

Test laboratory:

Task: *Matrix Operations Library*

Overview:

Develop a small C++ library to handle basic operations on matrices. This task will evaluate your understanding of C++ classes, operator overloading, and basic memory management.

Requirements:

1. *Matrix Class Implementation:*
 - a. Implement a Matrix class that encapsulates a two-dimensional array of doubles.
 - b. The class should manage its memory, correctly using new and delete for operations on the dynamic array.
2. *Constructors and Destructor:*
 - a. Implement a constructor that creates a matrix of a given size (rows and columns) and initializes all elements to 0.
 - b. Implement a destructor to properly free the allocated memory.
 - c. Include a copy constructor that performs a deep copy of the matrix data.
3. *Assignment Operator:*
 - a. Overload the assignment operator, ensuring it performs a deep copy and handles self-assignment safely.
4. *Basic Operations:*
 - a. add(const Matrix& other): Adds two matrices. Throw an exception if the sizes don't match.
 - b. multiply(const Matrix& other): Multiplies two matrices. Ensure matrix dimensions are compatible for multiplication, or throw an exception.
 - c. transpose(): Returns a new matrix that is the transpose of the original.
5. *Utility Functions:*
 - a. print(): A function to output the matrix to the console, for debugging purposes.
 - b. fill(double value): Fills the matrix with a specified double value.
 - c. identity(int size): Static method to create an identity matrix of a given size.
6. *Operator Overloading:*
 - a. Overload the + operator to allow adding matrices using the + syntax.
 - b. Overload the * operator for matrix multiplication.
 - c. Overload the [] operator to allow accessing elements directly (e.g., matrix[i][j]).
7. *Exception Handling:*
 - a. Handle invalid operations, such as addition or multiplication of matrices with incompatible dimensions, by throwing exceptions.

Evaluation Criteria:

- a. **Correctness:** The program must compile without errors and function as specified.
- b. **Memory Management:** Proper handling of dynamic memory without leaks.
- c. **Robustness:** The program should handle edge cases and potential errors gracefully.
- d. **Code Clarity:** Code should be well-documented and easy to understand.

Submission:

Provide a single .cpp file containing the implementation and a separate .cpp file with a main function that demonstrates the use of the Matrix class and tests all functionalities, including edge cases.